

Lab 12: C and LC-3 Programming

Overview

For each of the three problems below, write **two programs**: one in C and one in LC-3 assembly. The C version should use `printf` to display the result. The LC-3 version should store the result in a labeled memory location (e.g. `RESULT`) and verify it in the LC-3 Tools simulator.

Tools

- C: [OnlineGDB](#) — set language to **C**
- LC-3: LC-3 Tools simulator/assembler

Submission: Submit all six programs (three `.c` and three `.asm`) in Week_12 folder of your repo.

Program 1 — Absolute Value

Write a program that computes the **absolute value** of the integer `-7` and stores (or prints) the result.

C Requirements

- Declare an `int` variable initialized to `-7`
- Use an `if-else` statement to compute the absolute value
- Print the result with `printf`

Expected output:

```
Absolute value of -7 is 7
```

LC-3 Requirements

- Store the value `-7` in a memory location labeled `X` using `.FILL #-7`
- Use `LD`, `BRzp`, `NOT`, and `ADD` to compute the absolute value
- Store the final result in a memory location labeled `RESULT`
- Verify `RESULT` contains `7` in the simulator after `HALT`

Program 2 — Sum of Integers 1 to N

Write a program that computes the **sum of all integers from 1 to N**, where $N = 5$. The answer should be 15.

C Requirements

- Declare `int` variables for `n` (set to 5), `sum` (initialized to 0), and a loop counter
- Use a `for` loop to accumulate the sum
- Print the result with `printf`

Expected output:

```
Sum of 1 to 5 = 15
```

LC-3 Requirements

- Store `N = 5` in a memory location labeled `N` using `.FILL #5`
- Use a **count-down loop**: load `N` into a register, add the counter to the running sum each iteration, decrement the counter, and branch back while the counter is positive
- Store the final sum in a memory location labeled `SUM`
- Verify `SUM` contains `#15` (decimal 15, hex `x000F`) in the simulator

Program 3 — Find the Maximum

Write a program that finds the **largest value** in the following array of five integers:

```
{ 3, 7, 2, 9, 4 }
```

The answer is 9.

C Requirements

- Declare and initialize an `int` array of five elements
- Initialize `max` to the first element
- Use a `for` loop to compare each remaining element to `max`, updating `max` when a larger value is found
- Print the result with `printf`

Expected output:

```
Maximum value is 9
```

LC-3 Requirements

- Store the five values in consecutive memory locations using five `.FILL` directives under the label `ARR`
- Use `LEA` to get the base address of the array, then `LDR` with increasing offsets (or a moving pointer) to load each element
- Compare each element to the current maximum using subtraction (`NOT` + `ADD`) and a `BRnz` branch to skip the update
- Store the final maximum in a memory location labeled `MAX`
- Verify `MAX` contains `#9` in the simulator

Hint: To test if `R3 > R1`, compute `R3 - R1` and branch with `BRnz` to skip (meaning "if the result is not positive, don't update max").

Style requirements (apply to both C and LC-3):

- C: use `//` comments, initialize all variables, use `{}` braces on all blocks
- LC-3: include a header comment block (assignment, name, description, register usage), align labels/opcodes/operands/comments in consistent columns, comment most instructions